

Mastering TensorFlow: Day 5 - Building Your First Neural Network

Recap of Day 4

Yesterday, we focused on dataset handling and basic preprocessing techniques. Today, we'll apply those concepts to build a simple neural network.

```
import tensorflow as tf
import numpy as np
```

```
# This cell creates a simple dataset, similar to what was done on Day 4, to ensu
```

```
# Creating a simple dataset with NumPy
data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
labels = np.array([0, 1, 2])
```

```
# Converting the data to TensorFlow tensors
data_tensor = tf.constant(data, dtype=tf.float32)
labels_tensor = tf.constant(labels, dtype=tf.int32)
```

```
# Normalizing the data
normalized_data_tensor = data_tensor / 9.0
```

```
# Displaying the normalized data
print("Normalized Data Tensor:\n", normalized_data_tensor)
```

```
# Displaying the labels
print("Labels Tensor:\n", labels_tensor)
```

```
➦ Normalized Data Tensor:
tf.Tensor(
[[0.11111111 0.22222222 0.33333334]
 [0.44444445 0.55555556 0.66666667]
 [0.77777778 0.88888889 1.          ]], shape=(3, 3), dtype=float32)
Labels Tensor:
tf.Tensor([0 1 2], shape=(3,), dtype=int32)
```

```
# Building a simple neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=3, input_shape=[3]), # Input layer
    tf.keras.layers.Dense(units=3, activation='relu'), # Hidden layer
    tf.keras.layers.Dense(units=1) # Output layer
])

# Compiling the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Displaying the model summary
model.summary()
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Par
dense (Dense)	(None, 3)	
dense_1 (Dense)	(None, 3)	
dense_2 (Dense)	(None, 1)	

Total params: 28 (112.00 B)
 Trainable params: 28 (112.00 B)
 Non-trainable params: 0 (0.00 B)

```
# Training the model
model.fit(normalized_data_tensor, labels_tensor, epochs=10)
```

```
Epoch 1/10
1/1 ----- 1s 1s/step - loss: 1.7522
Epoch 2/10
1/1 ----- 0s 27ms/step - loss: 1.7233
Epoch 3/10
1/1 ----- 0s 28ms/step - loss: 1.6948
Epoch 4/10
1/1 ----- 0s 28ms/step - loss: 1.6666
Epoch 5/10
1/1 ----- 0s 28ms/step - loss: 1.6387
Epoch 6/10
1/1 ----- 0s 59ms/step - loss: 1.6112
Epoch 7/10
1/1 ----- 0s 27ms/step - loss: 1.5841
Epoch 8/10
1/1 ----- 0s 27ms/step - loss: 1.5573
Epoch 9/10
1/1 ----- 0s 28ms/step - loss: 1.5309
Epoch 10/10
1/1 ----- 0s 61ms/step - loss: 1.5048
```

<keras.src.callbacks.history.History at 0x7+d08d436290>

```
# Making predictions with the trained model
new_data = np.array([[2, 3, 4]]) / 9.0
predicted_label = model.predict(new_data)
print("Predicted Label for new data:", predicted_label)
```

1/1 ————— 0s 166ms/step
Predicted Label for new data: [[0.21870604]]

✓ Conclusion

Today, we built our first neural network using TensorFlow. We connected the dots between our dataset, preprocessing steps, and model training. Tomorrow, we'll dive deeper into model evaluation and performance metrics. Stay tuned! 🚀

Start coding or [generate](#) with AI.